1 Grundlagen der Kryptographie

1.1 Ziele und Grundlagen der Kryptographie

Kryptographie ist die Wissenschaft vom Schutz von Informationen. Die Disziplin lässt sich historisch und methodisch in zwei Phasen unterteilen:

- **Klassische Kryptographie (bis ca. 1970):** Das primäre Ziel war die **Vertraulichkeit**. Algorithmen basierten auf manuellen oder mechanischen Verfahren ohne mathematische Sicherheitsbegründung (*Security by Obscurity*). Sie sind anfällig für Brute-Force- und Häufigkeitsanalysen.
- **Moderne Kryptographie (ab ca. 1970):** Basiert auf schwer lösbaren mathematischen Problemen. Die Algorithmen sind öffentlich bekannt, nur der Schlüssel bleibt geheim.

Ziele der modernen Kryptographie

- **Vertraulichkeit (Confidentiality):** Nur autorisierte Empfänger können die Nachricht lesen.
- **Integrität (Integrity):** Nachrichten können nicht unbemerkt verändert werden.
- **Authentizität (Authenticity):** Die Identität des Absenders ist zweifelsfrei prüfbar.
- **Nicht-Abstreitbarkeit (Non-Repudiation):** Der Absender kann das Senden einer Nachricht nicht leugnen.

Motivation zur Verschlüsselung – Das Schlüsselverteilungsproblem: Sollen zwei Parteien (Alice und Bob) sicher über einen unsicheren Kanal kommunizieren, müssen sie sich auf einen geheimen Schlüssel einigen, ohne dass ein Angreifer (Eve) diesen abfängt. Dies ist das zentrale Problem der symmetrischen Kryptographie.

1.2 Klassische Verschlüsselungsverfahren

Ein formales Verschlüsselungsschema besteht stets aus Klartext (m), Chiffretext (c), einem geheimen Schlüssel (k) sowie Ver- und Entschlüsselungsfunktionen (E und D). Es gilt die **Korrektheitsbedingung**:

$$D_k(E_k(m)) = m$$

1.2.1 Caesar- und Shift-Chiffre

Die Shift-Chiffre ist eine **monoalphabetische Substitution**, bei der jeder Buchstabe des Alphabets um einen festen Wert k verschoben wird. Die Caesar-Chiffre ist ein Spezialfall mit $k = 3$.

Mathematische Formulierung der Shift-Chiffre

Buchstaben werden als Zahlen kodiert ($A = 0, B = 1, \ldots, Z = 25$). Die Operation erfolgt im Modulo n (Alphabetgröße, hier $n = 26$).

- Verschlüsselung: $E_k(x) = (x + k) \pmod{26}$
- Entschlüsselung: $D_k(y) = (y - k) \pmod{26}$

Angreifbarkeit: Sehr leicht brechbar.

- **Brute-Force:** Da es nur 25 sinnvolle Schlüssel gibt, können alle systematisch durchprobiert werden.
- **Häufigkeitsanalyse:** Typische Buchstabenverteilungen einer Sprache (z. B. das 'E' im Deutschen) bleiben im Chiffretext erhalten.
- **Known-Plaintext:** Kennt man einen Teil des Klartextes (z. B. eine Grußformel), lässt sich die Verschiebung k mit einer einfachen Subtraktion sofort berechnen.

1.2.2 Substitutionschiffre

Jeder Buchstabe wird durch einen anderen festen Buchstaben ersetzt (Permutation des Alphabets). Obwohl der Schlüsselraum mit ca. $4 \cdot 10^{26}$ (bei 26 Buchstaben) sehr groß ist, ist das Verfahren unsicher. *Warum?* Die Zuordnung ist statisch. Gleiche Buchstaben im Klartext erzeugen gleiche Buchstaben im Chiffretext. Die Methode ist daher extrem anfällig für **Häufigkeitsanalysen**. Eine große Anzahl an Schlüsseln bedeutet nicht automatisch Sicherheit.

1.2.3 Vigenère-Chiffre

Die Vigenère-Chiffre erweitert die Shift-Chiffre zu einem **polyalphabetischen** Verfahren. Es werden mehrere verschiedene Verschiebungen verwendet, abhängig von einem Schlüsselwort, das solange wiederholt wird, bis es die Länge des Klartextes erreicht.

Mathematische Formulierung der Vigenère-Chiffre

Für jeden Buchstaben an der Position i im Text und den korrespondierenden Schlüsselbuchstaben k_i:

- Verschlüsselung: $E_{k_i}(x_i) = (x_i + k_i) \pmod{26}$
- Entschlüsselung: $D_{k_i}(y_i) = (y_i - k_i) \pmod{26}$

Eigenschaften & Sicherheit: Gleiche Klartextbuchstaben werden je nach Position im Text unterschiedlich verschlüsselt. Dies verschleiert einfache Häufigkeitsmuster. Dennoch ist sie **nicht perfekt sicher**. Da das Schlüsselwort wiederholt wird, entsteht eine **Periodizität**. Ein Angreifer kann die Schlüssellänge ermitteln (Kasiski-Test) und den Text dann in entsprechende Spalten aufteilen. Jede Spalte entspricht dann wieder einer einfachen Shift-Chiffre und kann per Häufigkeitsanalyse gebrochen werden. *Fazit:* Je länger das Schlüsselwort (und je weniger Wiederholungen), desto sicherer das Verfahren.

1.3 One-Time Pad (OTP)

Das One-Time Pad ist der Grenzfall der Vigenère-Chiffre, bei dem das Schlüsselwort exakt so lang ist wie die Nachricht und **streng zufällig** gewählt wird.

One-Time Pad (OTP)

Jede Nachricht m der Länge n wird mit einem zufälligen Schlüssel $k \in \{0,1\}^n$ verschlüsselt, wobei das logische **Exklusive ODER (XOR, $\oplus$)** verwendet wird.

- KeyGen: Wähle k zufällig und gleichverteilt aus $\{0,1\}^n$
- Verschlüsselung: $c = m \oplus k$
- Entschlüsselung: $m = c \oplus k$

Verifikation der Korrektheit (da $k \oplus k = 0$): $D_k(E_k(m)) = (m \oplus k) \oplus k = m \oplus (k \oplus k) = m \oplus 0 = m$.

Perfekte Sicherheit (Theoretisch unknackbar): Ein Kryptosystem ist **perfekt sicher**, wenn der Chiffretext keinerlei Informationen über den Klartext preisgibt. Formal bedeutet das:

$$P(M|C) = P(M)$$

Beweisskizze: Da K komplett zufällig ist, ist die Wahrscheinlichkeit, dass ein spezifischer Chiffretext C aus einem Klartext M entsteht, genau die Wahrscheinlichkeit, dass $K = C \oplus M$ gewählt wurde ($P = 1/|K|$). Nach dem Satz von Bayes gilt $P(M|C) = \frac{P(C|M) \cdot P(M)}{P(C)}$. Da $P(C|M)$ konstant und unabhängig von M ist, folgt $P(M|C) = P(M)$. Ein Brute-Force-Angriff ist nutzlos, da jeder Entschlüsselungsversuch einen anderen validen Klartext erzeugt.

Probleme und Voraussetzungen des OTP:

1. **Schlüssellänge:** Der Schlüssel muss mindestens so lang sein wie die Nachricht, was für große Datenmengen ineffizient ist.
2. **Zufälligkeit:** Der Schlüssel muss echten Zufall aufweisen, nicht nur Pseudozufall.
3. **Einmaligkeit (Wiederverwendungsverbot):** Der Schlüssel darf **niemals** für eine zweite Nachricht verwendet werden!

Erklärung zum Wiederverwendungsverbot (Two-Time Pad Attack): Werden m_0 und m_1 mit demselben k verschlüsselt ($c_0 = m_0 \oplus k, c_1 = m_1 \oplus k$), kann der Angreifer beide Chiffretexte XOR-verknüpfen:

$$c_0 \oplus c_1 = (m_0 \oplus k) \oplus (m_1 \oplus k) = m_0 \oplus m_1 \oplus (k \oplus k) = m_0 \oplus m_1$$

Das Ergebnis verrät direkt die Unterschiede zwischen den beiden Klartexten. Wenn Teile von m_0 bekannt oder erratbar sind, lässt sich m_1 problemlos rekonstruieren.

1.4 Moderne Kryptographie und Formale Modelle

Das Kerckhoffs-Prinzip (1883)

Ein Kryptosystem muss auch dann sicher bleiben, wenn alle Details des Algorithmus öffentlich bekannt sind. Das einzige Geheimnis des Systems darf der **Schlüssel** sein.

Dies verhindert *Security by Obscurity*, fördert wissenschaftliche Überprüfbarkeit (offene Standards wie AES, RSA) und reduziert das Sicherheitsproblem auf ein isoliertes **Schlüsselmanagement**.

Kryptographische Primitive vs. Konstruktionen: Primitive sind grundlegende mathematische Bausteine (z. B. Blockchiffren, Hash-Funktionen). Konstruktionen kombinieren diese Primitive zu anwendbaren Protokollen (z. B. AES-CBC, HMAC, RSA-Signaturen). Sicherheit wird über formale Beweise erbracht: *Wenn das Primitiv sicher ist, ist auch die Konstruktion sicher.*

1.4.1 Formale Sicherheitsdefinitionen (Angreifermodelle)

Um Sicherheit mathematisch zu beweisen, definiert man **Spiele** zwischen einem Angreifer (Adversary) und einem Herausforderer (Challenger). Der Angreifer wählt in der Regel zwei gleich lange Nachrichten m_0 und m_1. Der Challenger wirft eine verdeckte Münze $b \in \{0, 1\}$ und gibt dem Angreifer den Chiffretext $c = E_k(m_b)$ zurück (den "Challenge-Ciphertext"). Der Angreifer muss nun raten (b'), ob m_0 oder m_1 verschlüsselt wurde. Das Schema ist sicher, wenn die Gewinnwahrscheinlichkeit des Angreifers signifikant nicht besser ist als Raten:

$$\Pr[b' = b] \leq \frac{1}{2} + \text{negl}(n)$$

Je nach Fähigkeiten des Angreifers unterscheidet man verschiedene Modelle:

- **IND-COA (Ciphertext-Only Attack):** Der Angreifer sieht ausschließlich Chiffretexte und muss daraus den Klartext oder Schlüssel ableiten (Standardangriff auf klassische Chiffren).
- **IND-KPA (Known-Plaintext Attack):** Der Angreifer kennt einige (Klartext, Chiffretext)-Paare und versucht daraus Muster oder den Schlüssel abzuleiten (z. B. bekannte Header-Formate wie "HTTP/1.1").
- **IND-CPA (Chosen-Plaintext Attack):** Der stärkste Angriff in der passiven Kategorie. Der Angreifer hat Zugriff auf ein **Verschlüsselungs-Orakel**. Er darf beliebige Klartexte seiner Wahl zur Verschlüsselung einreichen und analysieren, bevor und nachdem er den Challenge-Ciphertext erhält.
- **IND-CCA (Chosen-Ciphertext Attack):** Der Angreifer hat zusätzlich Zugriff auf ein **Entschlüsselungs-Orakel**. Er darf beliebige Chiffretexte zur Entschlüsselung einreichen, mit der einzigen Ausnahme, dass er nicht den Challenge-Ciphertext c selbst anfragen darf. Viele klassische Implementierungen (wie altes RSA-Padding) scheiterten an CCA-Angriffen (z. B. Bleichenbacher-Angriff).

Vernachlässigbare Funktion (negl)

Eine Funktion $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$ heißt **vernachlässigbar**, wenn sie für genügend große n schneller gegen null geht als der Kehrwert jedes beliebigen positiven Polynoms $p(n)$. Formal: Für jedes Polynom $p(n)$ existiert ein n_0, sodass für alle $n > n_0$ gilt:

$$f(n) < \frac{1}{p(n)}$$

Dies bedeutet in der Praxis: Ein polynomiell beschränkter Angreifer (Rechenleistung z. B. begrenzt auf $O(n^3)$) hat eine Erfolgswahrscheinlichkeit, die quasi null ist. Funktionen wie 2^{-n} sind vernachlässigbar, Funktionen wie $1/n^2$ sind es nicht.

Praktische Angriffsvektoren trotz Verschlüsselung: Verschlüsselung allein garantiert keine Sicherheit. Typische reale Schwachstellen sind:

- Schwache Primitive (z. B. MD5)
- Implementierungsfehler (Buffer Overflows, z. B. Heartbleed in OpenSSL)
- Seitenkanalangriffe (Timing-Angriffe auf Rechenzeiten)
- Schlüsselmanagementfehler (z. B. versehentliches Veröffentlichen privater Schlüssel auf GitHub)
- Veraltete Protokolle (z. B. WEP bei WLAN)

2 Kryptografie

Die Kryptografie wird in drei Hauptkategorien unterteilt:

1. Symmetrische Kryptografie
2. Hashfunktionen
3. Asymmetrische Kryptografie

2.1 Symmetrische Kryptografie

Symmetrische Kryptografie ist eine Menge von kryptografischen Protokollen, bei der derselbe geheime Schlüssel für die Ver- und Entschlüsselung von Daten verwendet wird.

Symmetrische Kryptosysteme

Ein symmetrisches Kryptosystem ist ein 5-Tupel (M, K, C, e, d) bestehend aus:

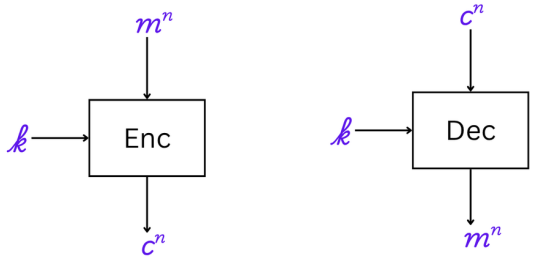
- einer Menge M von Klartexten,
- einer Menge K von Schlüsseln,
- einer Menge C von Chiffretexten,
- einer Verschlüsselungsfunktion $e : M \times K \to C$,
- einer Entschlüsselungsfunktion $d : C \times K \to M$,

so dass für alle Klartexte $m \in M$ und alle Schlüssel $k \in K$ gilt, dass $d(e(m, k), k) = m$.

2.1.1 Blockchiffren

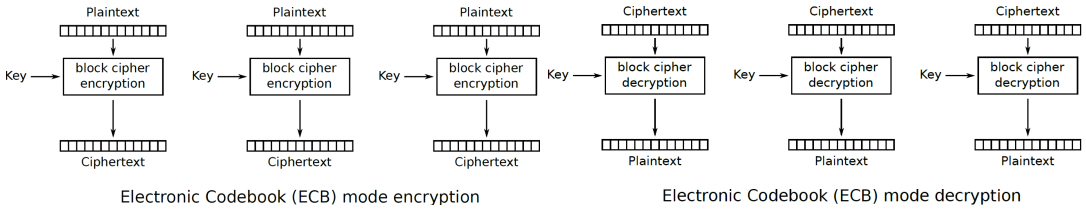
Definition

Blockchiffren sind Kryptosysteme, die nur Blöcke fester Länge verschlüsseln können.



- Ein Blockchiffre arbeitet auf einem Klartextblock der Länge b, um einen Chiffretextblock der Länge b zu erzeugen.
- Der gleiche Schlüssel kann mehrmals auf unterschiedliche Blöcke verwendet werden.
- Beispiele von Blockchiffren: AES, DES, 3DES, Serpent, Twofish, Blowfish, etc.

Electronic Code Book (ECB) Modus



Wenn die Blöcke nicht die Länge n haben, können trotzdem beliebige Nachrichten verschlüsselt werden, da eine **Auffüllfunktion** (Padding function) benutzt wird.

Bei vielen Padding-Verfahren wird *immer* ein Padding hinzugefügt, auch wenn die Nachricht bereits ein Vielfaches der Blocklänge n hat. Dies ist notwendig, damit die $unpad()$-Funktion eindeutig feststellen kann, wie viele Bytes entfernt werden müssen. Eine gute Auffüllfunktion sollte umkehrbar sein, d.h. es muss eine $unpad()$-Funktion geben mit $unpad(pad(x)) = x \quad \forall x \in M*$.

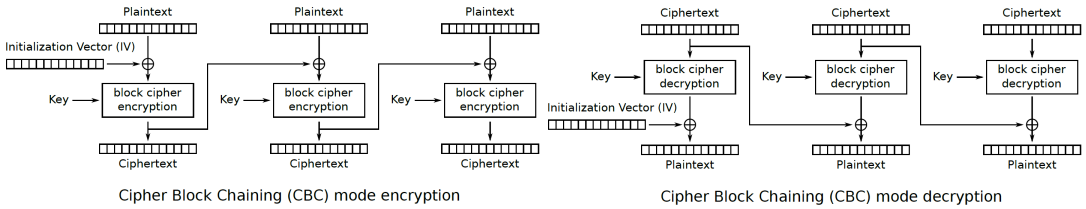
Vorteile:

- Unkomplizierte Bedienung. Jeder Block wird unabhängig bearbeitet.
- Parallelisierbarkeit von Ver- und Entschlüsselungsverfahren.
- Beschädigte Datenblöcke beeinflussen keine anderen Blöcke (Fehlertoleranz).

Nachteile:

- *Deterministisch:* Muster im Klartext sind sichtbar. Identische Klartextblöcke ergeben immer identische Chiffretextblöcke.
- *Keine Diffusion:* Kleine Änderungen im Klartext führen zu lokalisierten Änderungen im Geheimtext.

Cipher Block Chaining (CBC) Modus



Zur Formalisierung von CBC benötigen wir randomisierte Kryptosysteme. Der Zufallswert r wird hier als Initialisierungsvektor (IV) bezeichnet.

Randomisierte symmetrische Kryptosysteme

Ein randomisiertes symmetrisches Kryptosystem ist ein 5-Tupel (M, K, C, e, d) bestehend aus:

- einer Menge M von Klartexten,
- einer Menge K von Schlüsseln,
- einer Menge C von Chiffretexten,
- einer Menge R von Zufallswerten (z.B. IVs),
- einer Verschlüsselungsfunktion $e : M \times K \times R \to C$,
- einer Entschlüsselungsfunktion $d : C \times K \to M$,

(Anmerkung: Die Entschlüsselung d benötigt den IV r, dieser wird aber typischerweise als Teil des Chiffretextes C übermittelt und nicht als separater Zufallseingang für d selbst.)

Sei $r \in R$ der Initialisierungsvektor (IV). **Verschlüsselung**

$$e^*(x_0x_1 \dots x_n, k, r) = y_0y_1 \dots y_n \text{ mit } y_0 = e(x_0 \oplus r, k) \quad \text{und} \quad y_i = e(x_i \oplus y_{i-1}, k) \quad \text{für } i \geq 1$$

Entschlüsselung

$$d^*(y_0y_1 \dots y_n, k, r) = x_0x_1 \dots x_n \text{ mit } x_0 = d(y_0, k) \oplus r \quad \text{und} \quad x_i = d(y_i, k) \oplus y_{i-1} \quad \text{für } i \geq 1$$

- Zur Verschlüsselung muss ein Wert $r \in R$ (der IV) gewählt werden.
- Zufallswerte aus R (IVs) sind nicht geheim, sie können unverschlüsselt mit dem Chiffrat gespeichert und verschickt werden (meist als erster Block).
- Wir wollen $e(x, k, r^1) \neq e(x, k, r^2)$ für $r^1 \neq r^2$.
- Wichtig für die Sicherheit ist, dass der IV (Zufallswert r) **eindeutig** (nie doppelt für denselben Schlüssel) und **unvorhersagbar** ist.
- Muster im Klartext sind im Chiffrat nicht mehr erkennbar.
- Gleiche Klartextblöcke werden unterschiedlich verschlüsselt.
- Ein fehlerhafter Chiffreblock y_i führt nur zur fehlerhaften Entschlüsselung des aktuellen Blocks x_i und des unmittelbar nachfolgenden Blocks x_{i+1}.
- Verschlüsselung ist **nicht** parallelisierbar (sequenziell), Entschlüsselung ist parallelisierbar.

CBC Padding Oracle Attack CBC ist anfällig für Padding-Oracle-Angriffe

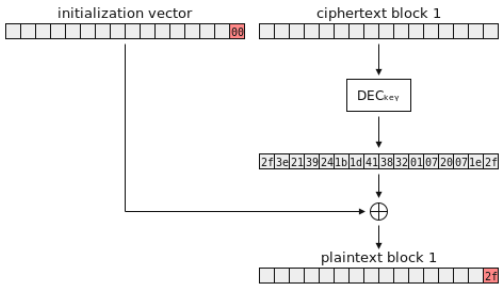
Ein solcher Angriff ermöglicht es, einen Geheimtext Schritt für Schritt zu entschlüsseln, ohne den Verschlüsselungsschlüssel zu kennen. Der Angreifer nutzt aus, wie ein Server auf fehlerhaftes Padding reagiert.

Der Angreifer:

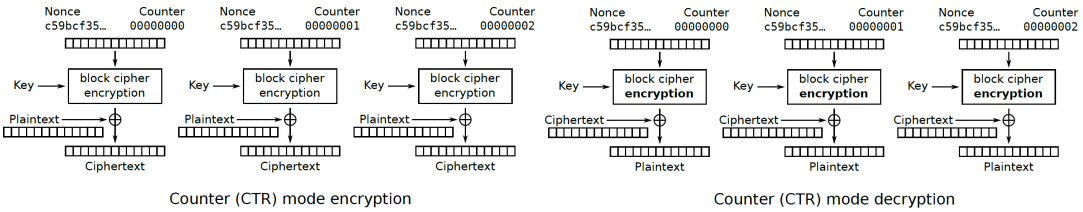
- hat keinen Zugriff auf den geheimen Schlüssel.
- ist in der Lage, gültige Chiffretexte abzufangen.
- ist in der Lage, modifizierte Versionen des Chiffretextes an das Orakel/Server zu senden und dessen Antworten zu beobachten.

Das Orakel (Server):

- muss ein überprüfbares Padding-Schema verwenden.
- muss dem Angreifer verraten, ob ein entschlüsselter Text ein gültiges (oder nicht) Padding hat. Dies kann geschehen durch:
  1. direkte Fehlermeldungen (z.B. HTTP 500) oder
  2. Side-Channel-Messungen (z.B. Unterschiede im Antwortverhalten).



Counter Mode (CTR) Modus



Um diesen Modus zu formalisieren, benötigen wir eine randomisierte Zählfunktion, die einen "Nonce" (Number used once) r verwendet.

- Ein randomisierter Zähler (Funktion $ctr(r, i)$) bildet einen Zufallswert (Nonce r) und eine natürliche Zahl i (Zähler) auf eine Bitkette fester Länge ab.
- Eine einfache Implementierung benutzt die Binärdarstellung der natürlichen Zahl (LSB- oder MSB-Kodierung) mit 0-Padding, konkateniert mit der Nonce.
- Ein randomisierter Zähler $\mathrm{ctr}(r, \cdot)$ sollte injektiv sein (für ein festes r). Man sollte die Periode (Wiederholung) so lang wie möglich wählen.
- Die Nonce r muss für denselben Schlüssel **nie** wiederverwendet werden.

Verschlüsselung:

$$e^*(x_0x_1 \dots x_n, k, r) = y_0y_1 \dots y_n \text{ mit } y_i = e(\mathrm{ctr}(r, i), k) \oplus x_i$$

Entschlüsselung:

$$d^*(y_0y_1 \dots y_n, k, r) = x_0x_1 \dots x_n \text{ mit } x_i = e(\mathrm{ctr}(r, i), k) \oplus y_i$$

Der CTR Modus unterscheidet sich stark von den vorher betrachteten Betriebsmodi:

- Ver- und Entschlüsselung nutzen beide die Verschlüsselungsfunktion e der Blockchiffre; die Entschlüsselungsfunktion d selbst wird nicht benötigt.
- Ver- und Entschlüsselung sind identisch (XOR mit dem Keystream).
- Die Berechnung des Keystreams $e(\mathrm{ctr}(r, i), k)$ ist unabhängig vom zu verschlüsselnden Text.
- Ver- und Entschlüsselung können vollständig parallelisiert werden.
- CTR ist eine One-Time-Pad-Konstruktion, bei der die Blockchiffre als Pseudozufallsgenerator (Keystream-Generator) dient.

Advanced Encryption Standard (AES)

- Blocklänge ist 128 bereits
- AES-Schlüssel können $128, 192,$ oder256 bits lang sein

Sicherheit

- AES ist sicher solange die Implementierung und dazugehörige Systeme richtig konfiguriert sind (s. CBC Padding Attack)

- Schwache Schlüssel und IV-Generierung kann die Sicherheit von AES gefährden
- Side-channel Angriffe wie cache-timing und power analysis können verwendet werden, um den Schlüssel abzuleiten

Gegenmaßnahmen

- Konstantzeit-Implementierung (gegen Timing Angriffe): Ausführungszeit von Code soll unabhängig von den verarbeiteten geheimen Daten sein
-

Stromchiffren Stromchiffren können beliebig lange Bitketten verschlüsseln. Dabei sind Klar- und Chiffretexte beliebiger Länge, nur der Schlüssel hat eine feste Länge. Aus dem Schlüssel wird ein pseudozufälliger Schlüsselstrom erzeugt. Pseudozufallszahlen hängen von ihren Startparametern ab (seed) ab - gleiche Parameter liefern gleiche Zufallszahlen Ver- und Entschlüsselung ist ein bitweise exklusives oder (XOR) mit dem Schlüsselstrom. Ein Kryptosystem heißt Stromchiffre, wenn es eine Funktion $keystream(x, z) = |x|$ gibt, so dass $e(x, y) = d(x, z) = x \oplus keystream(x, y)$. Die Funktion keystream nenn wir Schlüsselstromgenerator und ihren Funktionswert Schlüsselstrom.

- Keystream sollte ein Pseudozufallszahlengenerator sein
- Keystream kann unabhängig vom INhalt der ersten Variable sein, also $keystream(x_1, k) = keystream(x_2, k)$ für beliebige x_1 und x_2 mit $|x_1| = |x_2|$
- Falls der Schlüsselstrom sich wieder, ist die Stromchiffre nicht mehr sicher

ChaCha20 ist eine moderne Stromchiffre, die als Alternative zu AES entwickelt wurde.

2.2 Kryptografische Hashfunktionen

Eine Hashfunktion ist ein Algorithmus, der eine Eingabe beliebiger Größe in einen Hashwert mit einer festen Länge umwandelt. Hashfunktionen sind deterministisch, erlauben eine schnelle Berechnung und bieten Integritätsschutz (Änderung der Eingabe ändert den Hash) Eigenschaften einer Hashfunktion:

1. **Pre-Image Resistance**
   Bei gegebenem Hashwert h ist es rechnerisch unmöglich, die ursprüngliche Nachricht m zu finden, so dass $H(m) = h$.
2. **Second-Image Resistance**
   Bei gegebener Nachricht m_1 ist es rechnerisch unmöglich, eine andere Nachricht m_2 zu finden, die denselben Hashwert erzeugt, so dass $H(m_1) = H(m_2)$
3. **Collision Resistance**
   Es ist rechnerisch unmöglich irgendwelche zwei unterschiedlichen Nachrichten m_1 und m_2 zu finden, die denselben Hashwert erzeugen, so dass $H(m_1) = H(m_2)$

Hashfunktion	**Output**	**Sicherheit**	**Anwendung**
MD5	128 Bits	Unsicher	X
SHA-1	160 Bits	Unsicher seit 2017	X
SHA-256	256 Bits	Sicher	TLS/SSL, hashing, Blockchain
SHA-3/Keccak	224/256/384/512 Bits	Sicher	Ähnlich wie SHA-2 (aber langsamer ohne Hardware Unterstützung)

Table 1: *Vergleich von Hashfunktionen*

2.3 Asymmetrische Kryptografie

2.3.1 Grundlagen

Bei einem asymmetrischem Kryptosystem gibt es verschiedene Schlüssel.

1. Öffentliche Schlüsel werden frei für alle interessierten Mitredner veröffentlicht.
2. Eine geheime Nachricht muss erst mit dem öffenltichen Schlüssel verschlüsselt an den Empfänger zugeschickt werden.

Asymmetrische Kryptografie

Ein asymmetrisches Kryptosystem ist ein 7-Tupel $(M, K_s, K_p, K, C, e, d)$ bestehend aus
- einer Menge M von Klartexten,
- einer Menge K_s von geheimen/privaten Schlüsseln,
- einer Menge K_p von öffentlichen Schlüsseln
- einer Menge $K \subset K_s \times K_p$ von Schlüsselpaaren,
- einer Menge C von Chiffretexten,
- einer Verschlüsselungsfunktion $e : M \times K_p \to C$,
- einer Entschlüsselungsfunktion $d : C \times K_s \to M$,

so dass für alle Klartexte $m \in M$ und alle Schlüsselpaare $(s, p) \in K$ gilt, dass $d(e(m, p), s) = m$.

Prinzip:

- Verschlüsselung mit **öffentlichem Schlüssel** des Empfängers
- Entschlüsselung mit **privatem Schlüssel** des Empfängers
- Kein vorheriger Schlüsselaustausch nötig (im Gegensatz zu symmetrischer Kryptographie)

2.4 RSA-Kryptosystem

Idee

Das RSA-Kryptosystem (nach **Rivest, Shamir, Adleman**, 1977) ist das bekannteste Verfahren der asymmetrischen Kryptographie. Es basiert auf der Schwierigkeit, eine große Zahl $n = p \cdot q$ in ihre Primfaktoren zu zerlegen.

2.4.1 Schlüsselerzeugung

1. **Wähle zwei große Primzahlen** p, q mit $p \neq q$.
2. Berechne das **RSA-Modul**:
$$n = p \cdot q$$
3. Berechne die **Eulersche Totientfunktion**:
$$\varphi(n) = (p-1)(q-1)$$
Diese gibt die Anzahl der zu n teilerfremden Zahlen an.
4. Wähle den **Verschlüsselungsexponenten** e mit
$$1 < e < \varphi(n), \quad \gcd(e, \varphi(n)) = 1$$
(d. h. e und $\varphi(n)$ sind teilerfremd).
5. Berechne den **Entschlüsselungsexponenten** d als **modulares Inverses** von e:
$$d \cdot e \equiv 1 \pmod{\varphi(n)}$$
Dies geschieht mit dem **erweiterten Euklidischen Algorithmus**.

Beispiel: Erweiterter Euklidischer Algorithmus

Gegeben $e = 17$ und $\varphi(n) = 3120$:
Wir suchen d mit $17d \equiv 1 \pmod{3120}$.

$$\begin{aligned} 3120 &= 17 \cdot 183 + 9 \\ 17 &= 9 \cdot 1 + 8 \\ 9 &= 8 \cdot 1 + 1 \\ 8 &= 1 \cdot 8 + 0 \end{aligned}$$

Rückwärtseinsetzen:

$$1 = 9 - 8 = 9 - (17 - 9) = 2 \cdot 9 - 17 = 2(3120 - 17 \cdot 183) - 17 = 2 \cdot 3120 - 367 \cdot 17$$

Daraus folgt:

$$d \equiv -367 \equiv 2753 \pmod{3120}$$

Ergebnis: $d = 2753$.

Damit gilt:

$$\text{Öffentlicher Schlüssel: } (e, n), \quad \text{Privater Schlüssel: } (d, n)$$

2.4.2 Verschlüsselung und Entschlüsselung

- **Verschlüsselung:**
$$c = m^e \bmod n$$
- **Entschlüsselung:**
$$m = c^d \bmod n$$

Sicherheit von RSA:

- Basierend auf **Faktorisierungsproblem**: Schwierigkeit, n in p und q zu zerlegen
- Kenntnis von p, q oder $\varphi(n)$ ermöglicht Berechnung von d
- **Multiplikative Eigenschaft**: $(m_1^e \mod n) \cdot (m_2^e \mod n) \mod n = (m_1 m_2)^e \mod n \rightarrow$ problematisch für Sicherheit
- Durch **Quantencomputer** (Shor-Algorithmus) brüchbar
- p und q sollten groß und ähnlich groß sein (gleiche Bitlänge)

2.5 ElGamal-Kryptosystem

ElGamal Schlüsselerzeugung (Alice)

- Wähle eine **zyklische Gruppe** $G = (\mathbb{G}, \circ, e)$ mit großem Primzahlmodulus (z. B. $\mathbb{Z}_p^\times$) und einem **Erzeuger** g
- Wähle einen privaten Exponenten $a \in \{1, \ldots, \mathrm{ord}(g) - 1\}$ und berechne $A = g^a \bmod p$
- **Privater Schlüssel:** a
- **Öffentlicher Schlüssel:** (G, g, A)

Verschlüsselung (an Alice):

- Wähle zufällig $r \in \{1, \ldots, \mathrm{ord}(g) - 1\}$
- Berechne $R = g^r \bmod p$

- Berechne gemeinsamen Schlüssel $K = A^r = (g^a)^r = g^{ar} \bmod p$
- Berechne $C = (R,\, m \cdot K \bmod p)$ und sende C

Entschlüsselung (Alice):

- Berechne $K = R^a = (g^r)^a = g^{ra} \bmod p$
- Berechne das Inverse $K^{-1} \bmod p$
- Entschlüssele $m = C_2 \cdot K^{-1} \bmod p$

Sicherheit von ElGamal:

- Sicherheit basiert auf dem **Diskreten Logarithmusproblem (DLP)**: gegeben (g, g^a) ist a schwer zu bestimmen
- Angreifbar durch Quantencomputer (Shor-Algorithmus)
- **Probabilistisches Verfahren** durch Zufallswert r $\rightarrow$ semantisch sicher, wenn das **Decisional Diffie-Hellman-Problem (DDH)** schwer ist
- Aus einem gültigen Chiffrat (c_1, c_2) lässt sich leicht $(c_1,\, g \cdot c_2)$ für beliebiges $g \in G$ konstruieren — **nicht deterministisch**, daher keine Wiederverwendung von r

Hinweis: Bei allen Potenzoperationen und Multiplikationen muss stets das **Modulus** p angewendet werden. Der in der Vorlesung gezeigte Fehler (fehlendes mod$p$ bei K) wurde hier korrigiert.

- Kombination von asymmetrischer und symmetrischer Kryptographie
- Nachricht wird mit **symmetrischem Verfahren** verschlüsselt (schnell, effizient)
- Symmetrischer Schlüssel wird mit **asymmetrischem Verfahren** verschlüsselt (sicherer Schlüsselaustausch)
- Vorteile: Effizienz + Sicherheit, einfaches Teilen mit mehreren Empfängern
- Nachteil: Sicherheit von beiden Systemen abhängig

5. Digitale Signaturen

Zweck digitaler Signaturen

- **Authentizität**: Nachweis des Urhebers
- **Integrität**: Nachweis der Unveränderheit
- **Nicht-Abstreitbarkeit (Non-Repudiation)**: Unterzeichner kann Unterschrift nicht abstreiten

Rechtlicher Rahmen (eIDAS/VDG):

- **Einfache elektronische Signatur**: Keine besondere rechtliche Vermutung
- **Fortgeschrittene elektronische Signatur**: Eindeutige Zuordnung, hohes Vertrauen
- **Qualifizierte elektronische Signatur**: Rechtliche Gleichstellung mit handschriftlicher Unterschrift

2.5.1 RSA-Signaturen

RSA-Signaturverfahren

- Schlüsselgenerierung wie bei RSA
- Signieren: $s = (h(m))^d \mod n$
- Verifizieren: Teste ob $h(m) = s^e \mod n$
- **Hashfunktion h notwendig** zur Vermeidung von Angriffen

Digital Signature Algorithm (DSA)

DSA Parametergenerierung

1. Wähle Primzahl q (160/224/256 Bit)
2. Wähle Primzahl p (1024/2048/3072 Bit) mit $q \mid (p-1)$
3. Finde $g \in \mathbb{Z}_p^\times$ mit $\text{ord}(g) = q$
4. Parameter (p, q, g) sind öffentlich

DSA Schlüsselgenerierung und Signatur

- Privater Schlüssel: x mit $1 < x < q$
- Öffentlicher Schlüssel: $y = g^x \mod p$
- Signieren: Wähle k, berechne $r = (g^k \mod p) \mod q$, $s = k^{-1} \cdot (H(m) + r \cdot x) \mod q$
- Verifizieren: Berechne $w = s^{-1} \mod q$, $u_1 = H(m) \cdot w \mod q$, $u_2 = r \cdot w \mod q$, $v = (g^{u_1} \cdot y^{u_2} \mod p) \mod q$, akzeptiere wenn $v = r$

2.5.2 Sicherheitsbegriffe für Signaturen

Angreiferwissen

- **Key-Only Attack**: Nur öffentlicher Schlüssel bekannt
- **Known Signature Attack**: Nachricht-Signatur-Paare bekannt
- **Chosen Message Attack**: Signaturen für selbstgewählte Nachrichten erhältlich
- **Adaptive Chosen Message Attack**: Signaturen auch während Angriff erhältlich

Angreiferziele

- **Existential Forgery**: Neues gültiges Nachricht-Signatur-Paar
- **Selective Forgery**: Signatur für bestimmte neue Nachricht
- **Universal Forgery**: Signatur für beliebige Nachricht
- **Total Break**: Bestimmung des privaten Schlüssels

2.6 Schlüsselverteilung und Schlüsselaustausch

2.6.1 Schlüsselarten

- **Langzeitschlüssel**: Lange Gültigkeit (Monate/Jahre), häufig für Authentifizierung
- **Sitzungsschlüssel (Session Keys)**: Kurze Gültigkeit (eine Sitzung), reduziert Risiko bei Kompromittierung

Public Key Infrastructure (PKI)

Zertifikate

- Bestätigung der Zuordnung von öffentlichen Schlüsseln zu Identitäten durch vertrauenswürdige dritte Partei
- Enthalten: Öffentlicher Schlüssel, Name, Gültigkeitszeitraum, Aussteller, Signatur
- **X.509**: Hierarchisches, zentralisiertes System mit Root-Zertifikaten
- **Web of Trust**: Dezentrales System (PGP), gegenseitige Zertifizierung

2.6.2 Schlüsselaustauschprotokolle

Dolev-Yao-Angreifermodell

- Angreifer hat volle Kontrolle über Kommunikationskanal
- Kann: Abfangen, Verzögern, Unterdrücken, Ersetzen, Unter falscher Identität senden
- Kann **nicht**: Kryptographische Primitive brechen (perfekte Kryptographie angenommen)

Needham–Schroeder

- Symmetrische Version anfällig für Replay-Angriffe (veraltete Schlüssel)
- Asymmetrische Version sicherer, aber anfällig gegen aktive Angreifer ohne Authentifizierung

2.6.3 Diffie-Hellman-Schlüsselaustausch

Diffie-Hellman Protokoll

Dabei geht es um den Schlüsselaustausch um danach ein symmetrisches Kommunikationsverfahren anzuwenden. g^{ab} ist der gemeinsame symmetrische Schlüssel am Ende. Die öffentlichen Elemente sind: $g, p, g^a \mod p, g^b \mod p$

1. Einigung auf Primzahl p und Generator g von $\mathbb{Z}_p^\times$
2. A wählt a, sendet $g^a \mod p$ an B
3. B wählt b, sendet $g^b \mod p$ an A
4. A berechnet $(g^b)^a = g^{ab} \mod p$
5. B berechnet $(g^a)^b = g^{ab} \mod p$

Gemeinsamer Schlüssel: $K = g^{ab} \mod p$

Sicherheit:

- Basierend auf **Computational Diffie-Hellman Problem (CDH)**: Berechnung von g^{ab} aus g, g^a, g^b
- **Man-in-the-Middle-Angriff** möglich: Angreifer führt zwei separate DH-Protokolle
- Lösung: **Authenticated Diffie-Hellman** oder **Station-to-Station (STS)** Protokoll mit Signaturen

Station-to-Station (STS) Protokoll

- $A \to B$: g^a
- $B \to A$: g^b, $\{\text{sig}(sk_B, (g^a, g^b))\}_K$ mit $K = g^{ab}$
- $A \to B$: $\{\text{sig}(sk_A, (g^a, g^b))\}_K$

Signatur gewährleistet Authentizität und Integrität.

Logjam-Angriff (2015):

- Vorberechnung des diskreten Logarithmus für häufig verwendete Primzahlen
- Betraf 512/768 Bit, Abschätzung für 1024 Bit möglich
- Lösung: Verwendung größerer, individueller Primzahlen